

Coverage e Testing

M. Marulli

S. Remoli

Indice

1	Testing del software: concetti di base	2
1.1	Che cos'è un test	2
1.2	Schema Arrange-Act-Assert	2
2	JUnit (JUnit 5 / Jupiter)	2
2.1	Import essenziali	2
2.1.1	Annotazione @Test	2
2.1.2	Asserzioni (Assertions)	2
3	Organizzazione tipica del progetto	3
4	Coverage: che cosa misura realmente	3
4.1	Coverage come misura di esecuzione	3
4.2	Coverage non significa correttezza	4
4.3	If-statement e coverage	4
5	Coverage ottimistica	5
5.0.1	Esempio 1	5
5.0.2	Esempio 2	6
5.0.3	Esempio 3	6
6	Coverage pessimistica	7
6.0.1	Esempio 1	8
6.0.2	Esempio 2	8
6.0.3	Esempio sbagliato	9
7	Coverage impossibile	10
7.1	Casi non ammessi (banali)	10
7.2	Evento catastrofico dominio-specifico	10
7.2.1	Esempio 1	10
7.2.2	Esempio 2	11
7.3	Impossibile banale con HW/SW failure (non ammesso)	12

1 Testing del software: concetti di base

1.1 Che cos'è un test

Nel testing del software, un **test** è un pezzo di codice che esegue codice applicativo e controlla che il comportamento osservato corrisponda a ciò che è previsto. L'idea centrale è che il test non si limita a “*chiamare un metodo*”, ma deve anche verificare il risultato tramite un criterio esplicito (cioè un **oracolo**).

1.2 Schema Arrange–Act–Assert

Un test può essere visto come una procedura in tre momenti, spesso descritta con la forma **Arrange–Act–Assert**:

- **Arrange**: preparazione dello scenario (oggetti, input, stato);
- **Act**: invocazione del metodo da verificare;
- **Assert**: controllo del risultato con un'asserzione.

Questo schema non è una regola formale, ma una disciplina utile perché rende chiaro dove avviene la preparazione, dove avviene l'esecuzione e dove avviene la verifica.

2 JUnit (JUnit 5 / Jupiter)

JUnit è il framework di riferimento per il testing automatico in Java. Consente di scrivere test come normali classi Java, che vengono eseguite automaticamente da un runner (ad esempio quello integrato in IntelliJ o fornito da Maven/Gradle). Ogni test è un metodo che invoca codice applicativo e verifica il comportamento tramite asserzioni.

Un metodo di test viene riconosciuto come tale grazie all'annotazione **@Test**.

2.1 Import essenziali

Per scrivere un test JUnit 5 sono necessari principalmente due tipi di import.

2.1.1 Annotazione @Test

Il primo riguarda l'annotazione **@Test**, che indica al framework quali metodi devono essere eseguiti come test:

```
import org.junit.jupiter.api.Test;
```

2.1.2 Asserzioni (Assertions)

Il secondo riguarda le asserzioni, cioè i metodi che esprimono *l'oracolo* del test (ciò che ci si aspetta dal comportamento del codice). Le asserzioni più comuni sono **assertTrue**, **assertFalse**, **assertEquals**, **assertThrows**, e vengono importate tramite:

```
import static org.junit.jupiter.api.Assertions.*;
```

Esempio. Si consideri una classe applicativa:

```
public class ResoService {
    public boolean resoAccettato(int giorniDaConsegna, boolean sigillato) {
        return (giorniDaConsegna >= 0 && giorniDaConsegna <= 14) && sigillato;
    }
}
```

Un test che verifica un caso positivo (reso accettato) può essere scritto così:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ResoServiceTest {

    @Test
    void resoEntro14SigillatoAccettato() {
        ResoService s = new ResoService();           // Arrange
        boolean ok = s.resoAccettato(10, true);        // Act
        assertTrue(ok);                               // Assert
    }
}
```

Dal punto di vista di JUnit, ogni metodo annotato con `@Test` viene eseguito automaticamente. Il test passa se tutti gli `assert` risultano veri e non avvengono errori inattesi; il test fallisce se un `assert` risulta falso, oppure se viene lanciata un'eccezione non prevista.

3 Organizzazione tipica del progetto

In un progetto Java tipico (Maven/Gradle) si ha:

- `src/main/java` → codice applicativo (quello su cui si misura la coverage);
- `src/test/java` → test.

È importante perché la coverage viene calcolata sulle classi di `src/main/java`. Anche eventuali classi come `Main` o `App` (se presenti in `src/main/java`) contribuiscono alla coverage e devono essere eseguite dai test se si richiede il 100%.

4 Coverage: che cosa misura realmente

4.1 Coverage come misura di esecuzione

Nel contesto del testing, la **coverage** è una misura di esecuzione: indica quanta parte del codice applicativo viene effettivamente attraversata mentre girano i test. In altre parole, un report di coverage risponde a una domanda molto concreta: **quali linee del codice sono state eseguite almeno una volta?** Questa è l'informazione che interessa, perché una linea mai eseguita durante i test rappresenta un comportamento che non è mai stato osservato e che, di conseguenza, potrebbe nascondere errori non rilevati.

4.2 Coverage non significa correttezza

È essenziale ribadire un principio: la coverage **non misura la correttezza**. Una copertura elevata non garantisce l'assenza di bug; allo stesso modo, una copertura bassa non implica automaticamente la presenza di errori. La coverage è utile come indicatore di **zone non esplorate**, ma non sostituisce la qualità dell'oracolo del test (gli `assert`) e la scelta intelligente dei casi da provare.

Per capire questo punto, basta un esempio molto semplice:

```
int somma(int a, int b) {  
    return a + b;  
}
```

Un “test” senza `assert`:

```
@Test  
void testDebole() {  
    somma(2, 3);  
}
```

Questo test esegue la linea `return a + b;`, quindi produce coverage, ma **non verifica nulla**. Anche se `somma` fosse sbagliata, il test potrebbe comunque passare. Questo mostra chiaramente che coverage alta non significa automaticamente test buoni.

4.3 If-statement e coverage

Quando la coverage è intesa come **100% di linee coperte**, ogni linea di codice deve essere eseguita almeno una volta dai test. Un'istruzione `if` introduce una ramificazione: in una singola esecuzione si percorre un solo ramo, mentre le linee dell'altro restano scoperte se non si aggiungono ulteriori test.

Per esempio:

```
int f(int x) {  
    if (x >= 0) return 1;  
    return 2;  
}
```

Per coprire entrambe le linee di `return` servono almeno due test, uno con `x >= 0` e uno con `x < 0`. In generale, per un singolo `if` con due rami vale:

$$T_{\min} = 2.$$

Più in generale, se un frammento di codice contiene R rami mutuamente esclusivi (ad esempio una catena `if / else if / else`) e ciascun ramo contiene linee eseguite solo in quel caso, sono necessari almeno R test:

$$T_{\min} \geq R.$$

Se invece il metodo contiene N **decisioni indipendenti** (più `if` separati), nel caso peggiore i percorsi possibili crescono esponenzialmente:

$$\#\text{percorsi} = 2^N \Rightarrow T_{\min} \approx 2^N.$$

Da qui deriva l'osservazione fondamentale: aumentare gli `if` aumenta il numero di test necessari per raggiungere il 100% di coverage, motivo per cui, quando la copertura completa è un requisito, si preferiscono metodi con **flusso di controllo semplice** o ben suddivisi.

5 Coverage ottimistica

Nel senso usato nel corso, la **coverage ottimistica** è la situazione in cui tutte queste condizioni sono rispettate:

1. la coverage è 100% di linee coperte sul codice target (ad esempio una singola classe di logica);
2. i test risultano “ben scritti” (non sono test vuoti e non hanno errori);
3. nonostante ciò, nel codice esiste un **bug reale** che **non viene rilevato**.

Questo è possibile perché la line coverage garantisce soltanto che ogni linea è stata eseguita, non che sia stata validata in modo sufficiente rispetto a tutte le regole implicite del dominio.

Un test può essere formalmente corretto e sensato, ma incompleto rispetto alla specifica: copre il comportamento “normale”, **lasciando scoperta** la zona in cui il bug si manifesta.

5.0.1 Esempio 1

Consideriamo il seguente codice, con un bug:

```
public class AccessService {  
  
    // Regola attesa: accesso se admin oppure badge valido con prefisso "A-".  
    // BUG: prefisso sbagliato "B-".  
    public boolean canAccess(boolean isAdmin, String badge) {  
        return isAdmin || badge.startsWith("B-"); // BUG: dovrebbe essere "A-"  
    }  
}
```

Il seguente test:

```
class AccessServiceTest {  
  
    @Test  
    void adminPuoAccedere() {  
        AccessService s = new AccessService();  
        assertTrue(s.canAccess(true, "A-123")); // test corretto: un admin deve  
            ↪ passare  
    }  
}
```

La line coverage della classe è 100% (c'è una riga di **return**), il test è sensato e non errato, ma non valida la parte badge (**isAdmin=false**), quindi il bug resta.

Gli operatori logici *in corto circuito*, cioè **raddoppiati** (in particolare AND / **&&**) possono favorire la **coverage ottimistica** perché la seconda condizione viene valutata **solo se** la prima è vera. Di conseguenza, un test può eseguire la riga (quindi ottenere **100% di line coverage**) ma, scegliendo un input che rende la prima condizione falsa, non “*mettere mai alla prova*” la seconda parte, dove potrebbe essere presente un bug.

Esempio (senza codice). Un metodo stabilisce che un input è valido se “*input non nullo AND input ha un certo prefisso*”. Se i test controllano esclusivamente il caso “*input nullo → non valido*”, la riga risulta comunque coperta e il test è corretto rispetto al requisito verificato, ma il controllo sul prefisso non viene mai valutato. Di conseguenza, un eventuale errore nel controllo del prefisso resta nascosto ⇒ **coverage ottimistica**.

5.0.2 Esempio 2

Un altro esempio di coverage ottimistica:

```
public class TemperatureService {  
  
    // Regola attesa: conversione Celsius -> Fahrenheit:  $F = C * 9/5 + 32$   
    // BUG: usa 5/9 invece di 9/5 (errore di formula realistico).  
    public double toFahrenheit(double c) {  
        return c * (5.0 / 9.0) + 32.0; // BUG  
    }  
}
```

Il metodo `toFahrenheit` ha lo scopo di convertire una temperatura espressa in gradi Celsius in gradi Fahrenheit. La formula usata è matematicamente errata e produce risultati sbagliati per quasi tutti i valori di `c`, tranne in casi particolari.

Esempio: il seguente test:

```
class TemperatureServiceTest {  
  
    @Test  
    void zeroCorrispondeATrentadueF() {  
        TemperatureService s = new TemperatureService();  
        assertEquals(32.0, s.toFahrenheit(0.0), 1e-9); // test corretto e  
        ↪ standard  
    }  
}
```

Perché è ottimistica: a 0°C entrambe le formule danno 32, quindi il test è corretto ma non discriminante. Il bug emerge con qualsiasi altro valore (es. 100°C dovrebbe dare 212°F). Anche qui: coverage 100%, test sensato, bug nascosto.

5.0.3 Esempio 3

Consideriamo ora il seguente metodo che confronta due stringhe carattere per carattere usando un ciclo `for`:

```
public class StringCompareService {  
  
    public boolean equalsCharByChar(String a, String b) {  
        for (int i = 0; i < a.length(); i++) {  
            if (a.charAt(i) != b.charAt(i)) {  
                return false;  
            }  
        }  
        return true;  
    }  
}
```

Il bug è che il metodo assume implicitamente:

$$a.length() \leq b.length().$$

Quindi i seguenti test (due a causa dell'`if`):

```

class StringCompareServiceTest {

    @Test
    void stringheUgualiStessaLunghezza() {
        StringCompareService s = new StringCompareService();
        assertTrue(s.equalsCharByChar("abc", "abc"));
    }

    @Test
    void stringheDiverseStessaLunghezza() {
        StringCompareService s = new StringCompareService();
        assertFalse(s.equalsCharByChar("abc", "abd"));
    }
}

```

Tutte le linee vengono eseguite almeno una volta (quindi 100% line coverage) e i test passano senza problemi, non rilevando il bug reale.

6 Coverage pessimistica

La **coverage pessimistica** rappresenta la situazione **speculare** rispetto a quella ottimistica. Anche in questo caso si raggiunge il 100% di linee coperte, ma l'interpretazione del risultato dei test è opposta: il codice è corretto, mentre uno o più test falliscono, dando l'impressione che esista un bug quando in realtà il comportamento del metodo è conforme alla sua regola di business.

Le condizioni sufficienti per parlare di coverage pessimistica sono quindi:

- la coverage è al 100% (tutte le linee del codice applicativo vengono eseguite);
- il codice è corretto, ossia rispetta pienamente la specifica o la regola di business;
- almeno un test è **ragionevolmente sbagliato**, nel senso che assume un comportamento diverso da quello previsto dal metodo;
- il fallimento del test non è banale (non deriva da input palesemente assurdi), ma da una **violazione implicita della regola di business** su quello specifico dominio.

Il punto cruciale è proprio il concetto di errore **non banale**. Un test non è “ragionevolmente sbagliato” quando passa valori negativi a un metodo che chiaramente richiede numeri positivi, o quando si aspetta un risultato arbitrario. L'errore è invece *ragionevole* quando il tester fa un'assunzione **plausibile** ma errata sul comportamento del metodo, perché non conosce (o interpreta male) una regola di business incorporata nella logica.

Questo scenario è estremamente realistico nei sistemi software reali. Nella pratica, programmatore e tester sono spesso **ruoli distinti**, che lavorano in modo **disaccoppiato**. Il tester può non conoscere i dettagli interni dell'implementazione e può basarsi su una comprensione incompleta o approssimativa della specifica. Di conseguenza, può scrivere test che “*sembrano sensati*”, ma che in realtà contraddicono una regola esplicita o implicita del metodo.

Si parla di **coverage pessimistica** proprio perché il risultato dei test induce a una **valutazione eccessivamente negativa**: i test falliscono e suggeriscono la presenza di un bug, ma il problema non è nel codice, bensì nell'ipotesi fatta dal test.

6.0.1 Esempio 1

Si consideri la seguente regola di business:

Il reso è accettato solo se avviene entro 14 giorni dalla consegna e il prodotto è ancora sigillato.

Implementata dalla seguente classe:

```
public class ReturnService {  
  
    public boolean resoAccettato(int giorniDaConsegna, boolean sigillato) {  
        return giorniDaConsegna <= 14 && sigillato;  
    }  
}
```

Consideriamo il seguente test:

```
class ReturnServiceTest {  
  
    @Test  
    void resoEntroQuattordiciGiorniSempreAccettato() {  
        ReturnService s = new ReturnService();  
  
        // Assunzione plausibile ma sbagliata:  
        // entro 14 giorni il reso dovrebbe essere sempre accettato  
        assertTrue(s.resoAccettato(10, false));  
    }  
}
```

Il metodo ha banalmente 100% di coverage e il codice è corretto, ma il test fallisce in modo non banale.

6.0.2 Esempio 2

Consideriamo adesso la seguente regola:

Il prezzo finale è dato da prezzo base + IVA 22%, ma non può superare il tetto massimo di 200 euro.

Implementata da:

```
public class PriceService {  
  
    public double prezzoFinale(double base) {  
        double conIva = base * 1.22;  
        return Math.min(conIva, 200.0);  
    }  
}
```

Il seguente test:

```
class PriceServiceTest {  
  
    @Test
```

```

void prezzoConIvaSempreApplicata() {
    PriceService s = new PriceService();

    // Assunzione plausibile ma sbagliata:
    // il prezzo finale è sempre base + IVA, senza tetto massimo
    assertEquals(244.0, s.prezzoFinale(200.0), 1e-9);
}
}

```

Il tester ha fatto un'ipotesi ragionevole (*“si applica sempre l'IVA”*), ma incompleta, perché ignora il cap a 200 euro. Il test fallisce pur essendo il codice corretto con 100% di coverage.

6.0.3 Esempio sbagliato

Esempio **SBAGLIATO**. Si consideri la regola:

$$\text{prezzo} = \text{km} \times 1.5 \quad \text{con la precondizione} \quad \text{km} \geq 0.$$

Implementata dalla classe:

```

public class TicketService {

    public double prezzoBiglietto(double km) {
        if (km < 0) {
            throw new IllegalArgumentException();
        }
        return km * 1.5;
    }
}

```

Tuttavia il seguente test:

```

class TicketServiceTest {

    @Test
    void prezzoPerChilometriNegativi() {
        TicketService s = new TicketService();

        // ERRORE BANALE:
        // km negativi non hanno alcun significato nel dominio
        assertEquals(-15.0, s.prezzoBiglietto(-10.0));
    }
}

```

Questa **NON** è una coverage pessimistica perché il fallimento è banale:

- il test usa un input fuori dominio ($\text{km} < 0$);
- il metodo dichiara esplicitamente che quel caso non è ammesso;
- non c'è nessuna assunzione plausibile o ambigua;

7 Coverage impossibile

Si parla di **coverage impossibile** quando non è possibile raggiungere il 100% di coverage, non per negligenza, non per test mancanti, e non perché esiste **codice morto**, ma perché una parte del codice è legata a condizioni che, pur essendo logicamente sensate e realistiche, sono di fatto irrealizzabili nel contesto di test.

Il punto chiave è proprio questo: la coverage è inferiore al 100% **in modo non banale**.

7.1 Casi non ammessi (banali)

Non è ammesso, in questo concetto, usare:

- codice non raggiungibile per costruzione (*dead code*);
- `if (false);`
- rami logicamente impossibili per definizione matematica.

7.2 Evento catastrofico dominio-specifico

La coverage impossibile nasce invece quando il codice gestisce eventi eccezionali o catastrofici, che:

- hanno senso dal punto di vista del dominio,
- sono corretti dal punto di vista progettuale,
- ma non possono essere riprodotti in un test automatico realistico.

7.2.1 Esempio 1

Si consideri un sistema che monitora un impianto industriale. Una regola di sicurezza può essere:

Se viene rilevata una concentrazione di gas nervino superiore a una soglia critica, il sistema entra in modalità di emergenza totale.

Questa condizione è realistica e corretta dal punto di vista del dominio, ma non può essere riprodotta in un test senza eventi catastrofici.

```
public class SafetyService {  
  
    public boolean emergencyShutdown(double gasLevel) {  
        if (gasLevel > 10_000) {    // soglia catastrofica  
            return true;  
        }  
        return false;  
    }  
}  
  
class SafetyServiceTest {  
  
    @Test
```

```

void livelloGasNormale() {
    SafetyService s = new SafetyService();
    assertFalse(s.emergencyShutdown(50.0));
}
}

```

Il ramo “catastrofico” non è dead code ma non è realisticamente testabile. La coverage sarà necessariamente $< 100\%$, e questo non indica alcun problema.

7.2.2 Esempio 2

Si immagini un sistema che:

- riceve un insieme di password candidate (ad esempio da un file di import, da un dispositivo, da un recovery),
- le scorre per trovare la prima password valida,
- ma deve anche gestire un caso catastrofico: *ambient compromise*, cioè una situazione di sicurezza estrema (es. rilevazione di gas nervino / attacco fisico al data center / manomissione ambientale) che impone blocco immediato.

La condizione catastrofica è realistica nel dominio della sicurezza, ma impossibile da riprodurre in un test unitario “normale” senza simulazioni estreme. Quindi quel ramo tende a restare scoperto.

```

public class PasswordService {

    // Ritorna true se trova almeno una password valida.
    // In caso di evento catastrofico (compromissione ambientale), blocca
    ↪ subito.
    public boolean anyValidPassword(String[] candidates) {
        for (String p : candidates) {
            if (SecuritySensors.catastrophicCompromise()) {
                // Evento catastrofico: blocco immediato
                throw new SecurityException("Catastrophic atomic compromise
                ↪ detected");
            }

            if (p != null && p.length() >= 12) {
                return true;
            }
        }
        return false;
    }
}

```

Il metodo:

- scorre l’array `candidates`;
- se rileva una compromissione catastrofica, interrompe tutto con eccezione;
- altrimenti, considera “valida” la prima password non nulla con lunghezza ≥ 12 ;

- se nessuna è valida, restituisce **false**.

La coverage è impossibile perché il ramo:

```
if (SecuritySensors.catastrophicCompromise()) {  
    throw new SecurityException(...);  
}
```

non è dead code: è raggiungibile in teoria, ma dipende da un evento estremo e non riproducibile durante test ordinari (es. un sensore fisico di compromissione per un attacco nucleare).

7.3 Impossibile banale con HW/SW failure (non ammesso)

I casi banali di impossible coverage legati a *hardware/software failure* sono quelli in cui una parte del codice gestisce un guasto esterno “sensato”, ma non riproducibile in modo naturale dentro un normale unit test. Il codice è corretto, solo che la condizione di errore dipende dall’ambiente (DB, rete, disco, OS, hardware). Questi casi però sono considerati banali e quindi errati.

Di seguito due esempi di impossible coverage **BANALI** e quindi da non riprodurre:

1. **Database pieno / quota esaurita.** Esempi: tablespace pieno, storage esaurito, limiti di quota. È realistico in produzione, ma nei test è difficile riempire “davvero” lo storage o riprodurre l’errore in modo affidabile.
2. **Disco pieno / I/O failure.** Esempi: `IOException`, permessi negati, filesystem in sola lettura, corruzione. Il codice gestisce il fallimento di `write()` o `flush()`, ma nei test normali non si porta un filesystem a “disk full” per coprire quella linea.

Sono casi banali proprio perché non dipendono dal dominio specifico dell’applicazione, ma da infrastruttura e ambiente di esecuzione. Non codificano una regola di business dell’app: sono condizioni “di contorno” (risorse finite, I/O, servizi esterni) che esistono indipendentemente da cosa l’app stia facendo.

Un sensore “gas nervino” non è un caso banale perché:

- è **dominio-specifico**: ha senso in un contesto preciso (impianto industriale, sicurezza militare, laboratorio chimico, ecc.);
- è una **regola funzionale** dell’applicazione: quel ramo esiste perché l’app deve implementare una policy di sicurezza legata a quel fenomeno;
- è “impossibile” per motivi **semantici del dominio**: per coprire quel ramo serve produrre un evento catastrofico reale o altamente straordinario, non semplicemente stressare una risorsa.